Solving Tiny Hanamikoji: beliefs, regret, and equilibrium

CS6208 Homework 2 | 28 October 2026

1. Problem and design

I solve one round of Tiny Hanamikoji with neutral initial favors and a uniformly chosen starting player. Payoff is +1 for winning, -1 for losing, and 0 for an unresolved round. Hidden hands and reserves make decisions information-set dependent. I implemented the guided posterior, best-response, and CFR tasks, then investigated which CFR modifications improve convergence. A sequence-form LP supplies an independent reference and the submitted strategies.

The default tree has 2,757 information sets and 3,672 sequences per player. The supplied payoff map contains 12,832 sequence pairs, of which 2,816 have nonzero payoff. I keep the supplied treeplex file unchanged. Its sequence ordering determines compatibility with the strategy loader.

2. Beliefs and best responses

For target information set I, my posterior weight at a consistent history h is chance reach times opponent reach. I sum weights for identical state keys, then normalize. My own reach cancels under perfect recall: histories in I share my previous information sets and actions. A mismatched observation prefix cannot recover later, so I prune it. Zero-probability branches are omitted; impossible conditioning events return an empty dictionary. A copied initial state and finally/undo blocks preserve the caller's state.

Let x and y be realization plans and A the chance-weighted payoff matrix. Rewards are Ay for Player 1 and $-A^{T}x$ for Player 2. My best-response dynamic program traverses information sets from children to parents, maximizes each local action value, and adds it to the parent sequence. All child information sets sharing a sequence contribute; selecting only one would lose possible future observations.

SPR(x,y) = $\max_{x'} x'^{T}Ay - \min_{y'} x^{T}Ay'$. I use the full gap, without division by two. It equals the sum of unilateral deviation gains and bounds each player's incentive to deviate.

3. CFR implementation

I use full-tree counterfactual regret minimization [1]. Backward propagation computes continuation action values q(I,a), the behavioral expectation v(I), and regret increment q(I,a)-v(I). Regret matching normalizes positive cumulative regrets, using a uniform distribution when none is positive. Opponent and chance reach are already in the reward vector; my prefix reach must not multiply these regrets.

I average realization plans, not raw behavioral probabilities. This weights each information-set strategy by its own reach. Independent information sets at the same own-action depth are processed in NumPy batches; repeated parent indices use accumulation. Payoffs are refreshed between solve calls, and continuation and reset are supported.

4. Controlled convergence experiment

My hypothesis is that alternating updates, regret clipping, and reduced weight on early strategies explain different parts of the speedup. I therefore compare four configurations: CFR-S (simultaneous, uniform average), CFR-A (alternating, uniform), CFR+-U (alternating RM+, uniform), and CFR+-L (alternating RM+, linear average). RM+ updates stored regret as max(R+r,0), following [2]. Linear weights t affect only the strategy average; this is not linear weighting of regret increments.

All configurations start from zero regrets and use complete weighted chance enumeration for 2,000 iterations. Alternation updates P1 first, then P2 against refreshed P1; both pre-update plans enter the average. This explicit averaging convention is shared across the ablations. SPR is checked every 25 iterations, with no early stopping during measurement.

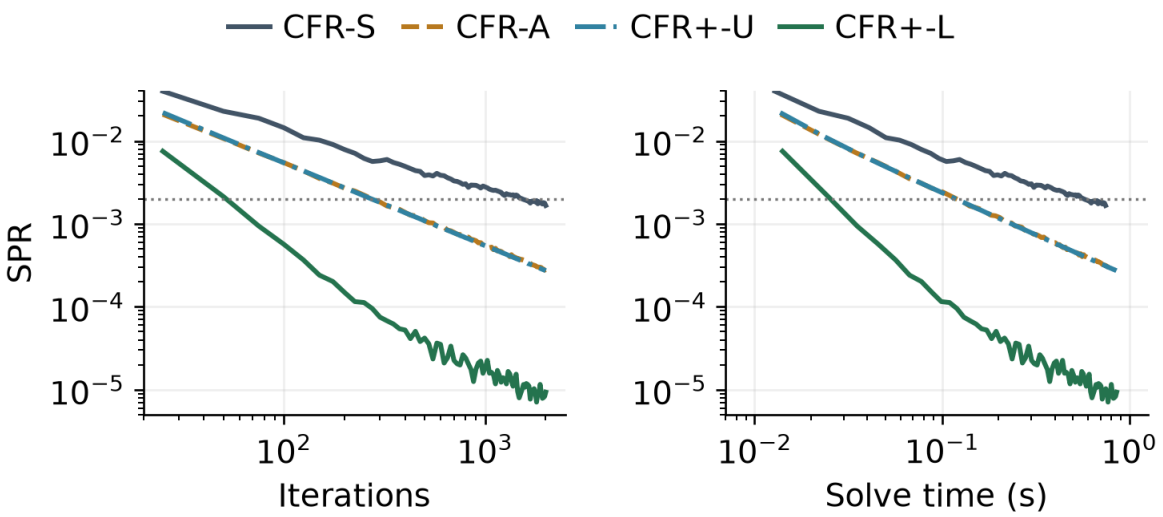


Figure 1. Default-game SPR against iterations and solve time for the first timing repetition. The dotted line is the assignment's illustrative target, 0.002. Time includes exact periodic SPR checks; tree construction and solver construction are excluded from the plotted solve clock.

Method	SPR at 2,000	First target check	Median time (s)
CFR-S	0.00167	1550	0.752
CFR-A	0.00028	300	0.849
CFR+-U	0.000272	275	0.857
CFR+-L	9.34e-06	75	0.860

Table 1. Three timing repetitions with rotated method order. Table time includes solver construction, excludes the shared tree build, and covers all 2,000 iterations. A first checkpoint crossing need not be the earliest exact crossing or remain below the target thereafter.

5. What accounts for the improvement?

At 2,000 iterations, alternation reduces the default gap by about 5.95 times relative to simultaneous CFR. Clipping adds only a 2.8% reduction when both use uniform averaging. Linear averaging then reduces the clipped solver's gap by about 29.1 times, yielding a total 178-fold reduction relative to CFR-S. The two clipped variants follow identical regret trajectories; the final-average difference isolates weighting, rather than a different self-play path.

The target is first observed at iteration 75 for CFR+-L versus 1,550 for CFR-S. Their first-repetition solve clocks are 0.0352 s and 0.579 s. These are checkpoint measurements, not precise hitting times. Alternation costs slightly more per iteration but improves accuracy substantially. Full timing ranges are retained in the raw results.

6. Independent sequence-form LP

I also solve the compact zero-sum LP [3]. Let $Ex=e$ and $Fy=f$ encode flow conservation: the empty sequence has weight one and each information set's action weights sum to its parent weight. Dualizing the opponent's optimization gives the following pair:

P1: maximize $f^{T}q$ subject to $Ex=e$, $x \geq 0$, $F^{T}q \leq A^{T}x$.
P2: minimize $e^{T}p$ subject to $Fy=f$, $y \geq 0$, $E^{T}p \geq Ay$.

The dual vectors p and q are unrestricted. I build sparse matrices and solve both programs with SciPy/HiGHS, checking success, primal and dual residuals, and the independent best-response gap. SciPy is imported inside the optional LP helper, so it is not required for the grader-facing functions.

The default LP value is numerically zero. Maximum primal residual is 3.33e-16; maximum dual residual is 2.60e-18. The LP objective gap is zero to floating-point precision. Its independent SPR is 8.67e-18. I submit these LP plans through the prescribed save_agent API; reloaded SPR remains 8.67e-18. Conversion changes realization weights by at most 5.55e-17. This is a numerical certificate, not symbolic exact arithmetic.

LP assembly plus both solves takes 0.0850 s at the median. The first LP run takes 0.514 s because it includes lazy optimizer initialization; the next two take 0.0844 and 0.0850 s. Only CFR receives an explicit two-iteration warm-up. Tree construction takes 0.933 s and is excluded from both comparisons. At this scale, preprocessing dominates an accurate LP solve; the experiment does not establish a large-game scaling advantage.

7. Robustness and interpretation

I repeat the same four configurations without tuning on two single-round variants: P1 starts from neutral favors; and P1 starts already controlling G3, with G1 and G2 neutral. Their treeplexes have 1,297/1,460 information sets and 1,902/1,771 sequences. These are changes in information and terminal rewards, not tests of a larger card game.

Method	Neutral, random start	Neutral, P1 start	P1 start, G3 held
CFR-S	0.00167	0.00167	0.000685
CFR-A	0.00028	0.000271	0.000309
CFR+-U	0.000272	0.000259	0.000275
CFR+-L	9.34e-06	1.19e-05	8.59e-07
LP value	0.000000	0.019048	0.057143

Table 2. Final average SPR at 2,000 iterations and independent LP values. CFR+-L is best among the four averages in all three cases. Positive values measure P(win)-P(loss) in this round, not win probability or the value of a multi-round match.

The default value is zero because the chance choice of starting player restores player symmetry. With P1 fixed to start, the LP value is about 0.01905; retaining G3 increases it to 0.05714. Equality of self-play payoff with the value does not certify an equilibrium: default CFR-S has near-zero self-play payoff even while exploitable. I therefore measure SPR directly.

Averaging is not uniformly better at every finite time. Default CFR-S has current-iterate SPR 0.11265 versus average SPR 0.001665. Default RM+ has current SPR 0.001656 versus linear-average SPR 0.00000934. In the G3-held game, alternating current strategies reach numerical equilibrium while averages still retain earlier mistakes. This explains why current payoff or a single current strategy is an unreliable stopping rule across games.

Exact evaluation against fixed opponents

Opponent	Nash as P1	Nash as P2
Random	+0.0945	+0.0904
Greedy	+0.1302	+0.1325
Balanced	+0.0663	+0.0587
CardCounting	+0.0411	+0.0373
Adaptive	+0.1127	+0.1143

Table 3. Expected payoff to the submitted policy, computed by exact chance-weighted evaluation. No Monte Carlo sampling error is involved. Equilibrium protects against all opponents, but need not maximize exploitation of a particular heuristic. The two seats may use different members of a nonunique equilibrium set.

8. Verification and reproducibility

All 21 tests pass. Six posterior tests include analytic hidden-card probabilities 1/5, 1/5, 3/5, a reserve-signal posterior of 1/2, 1/2, both seats, late own actions, impossible events, and state restoration after exceptions. A separate joint-law oracle includes both players' reach, independently checking the cancellation argument.

Fifteen solver tests include enumeration of all 32 pure plans of a branching treeplex, full Tiny best responses against separate LPs, and sequence payoff against live-tree traversal. The matrix game [[2,-1],[-1,1]] checks equilibrium (0.4,0.6), value 0.2, and reward signs; positive and negative payoff shifts check free dual variables.

Further checks cover flow conservation, RM+ nonnegativity, branching-game convergence, continuation/reset in all four configurations, zero iterations, changed-payoff cache refresh, and core imports with SciPy blocked. All four notebooks execute. The protected treeplex hash matches the starter. Export checks cover array shape, player ID, finite/nonnegative normalized probabilities, and reloaded SPR.

Measurements use Python 3.12.14, NumPy 2.5.4, SciPy 1.18.1, the Python engine, an Apple M4 with 16 GiB RAM, and macOS 15.6. Three repetitions measure timing only: full enumeration gives identical numerical trajectories. Peak process RSS is 128.6 MiB, not a per-solver comparison. The task4 folder retains source hashes, histories, certificates, tests, and a dependency lock.

I reproduce all results with the command sequence in task4/README.md. The experiment runner regenerates measurements and submitted strategies; the report builder and supplied packager produce the final PDF and ZIP.

9. Limits and next steps

This study supports a practical benefit from alternation and strategy averaging on small, fully enumerated games. It does not establish a better asymptotic convergence rate, scalability to Medium/Full Hanamikoji, or a solution to the multi-round game. My policy table covers only neutral first rounds; any fallback in a later round has no equilibrium guarantee. The next useful experiment would measure enumeration growth and sampled counterfactual updates under a fixed memory budget, rather than tune more parameters on Tiny.

References

[1] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione. *Regret Minimization in Games with Incomplete Information.* NeurIPS 20, 2007. Proceedings paper.

[2] O. Tammelin. *Solving Large Imperfect Information Games Using CFR+*. 2014. arXiv:1407.5042.

[3] B. von Stengel. *Efficient Computation of Behavior Strategies.* Games and Economic Behavior 14(2), 220-246, 1996. doi:10.1006/game.1996.0050.